

C.O.P.P.E.R

Coordinated Operational Pipeline for Parallel
Execution and Routing

Cloud GPU Training & Model Deployment Guide

Version 2.0 | July 2026

50 Specialist Agents + 1 Orchestrator

Optimized for RTX 5050 (8GB VRAM) + Ollama

*Covers: Vast.ai | RunPod | Google Colab | Local RTX 5050
LoRA Training | GGUF Quantization | Ollama Integration*

Table of Contents

1. Overview & Architecture
2. Cloud GPU Provider Comparison
3. Step-by-Step: Training on RunPod
4. Step-by-Step: Training on Vast.ai
5. Step-by-Step: Training on Google Colab (Free)
6. Local Training on RTX 5050 8GB
7. Converting LoRA Adapters to GGUF (Q4)
8. Loading Models into Ollama
9. Integrating Models into the COPPER Project
10. Troubleshooting
11. Cost Summary & Recommendations

COPPER -- Cloud GPU Training & Model Deployment Guide

Version 2.0 * July 2026

This guide covers everything from renting a GPU to running your fine-tuned 50-agent fleet inside the COPPER application.

Table of Contents:

- 1. Overview & Architecture**
 - 2. Cloud GPU Provider Comparison (Cheapest Options)**
 - 3. Step-by-Step: Training on RunPod (Recommended)**
 - 4. Step-by-Step: Training on Vast.ai (Cheapest)**
 - 5. Step-by-Step: Training on Google Colab (Free)**
 - 6. Local Training on RTX 5050 8GB (ROG Strix G16 2025)**
 - 7. Converting LoRA Adapters to GGUF (4-bit Quantized)**
 - 8. Loading Models into Ollama**
 - 9. Integrating Models into the COPPER Project**
 - 10. Troubleshooting**
 - 11. Cost Summary & Recommendations**
-

Section 1: OVERVIEW & ARCHITECTURE

COPPER has 50 specialist sub-agents + 1 orchestrator (COPPER itself). Each agent is fine-tuned separately using QLoRA (Quantized Low-Rank Adaptation) on a base model (e.g., Mistral-7B, Qwen2.5-3B, etc.).

The training pipeline:

- 1. Generate dataset -> CPU only, runs on any machine
- 2. Fine-tune (QLoRA) -> Requires a GPU (cloud or local)
- 3. Merge LoRA adapter -> CPU only
- 4. Quantize to GGUF -> CPU only (RAM-heavy)
- 5. Load into Ollama -> Runs on your laptop GPU

Files involved:

- * scripts/powershell/agent_configs.py -> Agent definitions
- * scripts/generate_all_agents.py -> Dataset generator
- * scripts/maintenance/copper_orchestrator_dataset_gen.py
- * scripts/finetune_agent.py -> QLoRA trainer
- * scripts/launch_finetune_all.sh -> Batch runner

Section 2: CLOUD GPU PROVIDER COMPARISON

Provider	RTX 3090 (24GB)	RTX 4090 (24GB)	A100 80GB	Notes	
Vast.ai	\$0.15-0.25/hr	\$0.30-0.45/hr	\$0.90-1.60/hr	CHEAPEST. Spot	
				marketplace. Use	
				"verified" hosts.	
RunPod	\$0.25-0.39/hr	\$0.34-0.69/hr	\$1.39-1.49/hr	BEST UX. Per-second	
				billing. Free	
				egress. Templates.	
Google Colab	Free (T4 15GB)	N/A	\$10/mo Pro	FREE tier works	
			(A100 access)	for 3B models.	
				Session timeouts.	
Lambda Labs	N/A	N/A	\$1.29-2.06/hr	Best stability.	
				Often out of stock.	

TOTAL COST ESTIMATE (all 50 agents + COPPER orchestrator):

- * Vast.ai RTX 3090: ~\$3.50 - \$6.00 total
- * RunPod RTX 4090: ~\$6.50 - \$10.00 total
- * Google Colab Free: \$0 (but requires patience and session management)

Section 3: STEP-BY-STEP -- TRAINING ON RUNPOD (Recommended)

STEP 1: Create an account at <https://runpod.io>

Add a payment method (credit card or crypto).

STEP 2: Deploy a Pod

- * Go to "Community Cloud" for cheapest rates
- * GPU: Select "RTX 4090 24GB" (~\$0.34/hr)

- * Template: Choose "RunPod PyTorch 2.x" (CUDA pre-installed)
- * Disk: 50 GB container disk
- * Click "Deploy"

STEP 3: Connect to the Pod

- * Click "Connect" -> "Start Web Terminal" (or use SSH)

STEP 4: Upload your COPPER project

Option A (Git):

```
git clone https://github.com/YOUR_USERNAME/COPPER.git
cd COPPER
```

Option B (Upload):

- * Zip your project locally
- * Use RunPod's file browser to upload
- * unzip COPPER.zip && cd COPPER

STEP 5: Install dependencies

```
pip install -q -r requirements_finetune.txt
```

STEP 6: Generate all datasets (CPU-only, takes ~2 minutes)

```
python generate_all_agents.py --size 1500 --outdir ./dataset
python copper_orchestrator_dataset_gen.py --size 2500 --outdir ./dataset/COPPER
```

STEP 7: Start training inside tmux (so SSH drops don't kill it)

```
tmux new -s copper-train
bash launch_finetune_all.sh 24gb
```

To detach: press Ctrl+B, then D

To reconnect: tmux attach -t copper-train

STEP 8: Monitor training

In a second terminal or tmux pane:

```
watch -n 2 nvidia-smi
```

Training loss is logged to stdout every 25 steps.

Expected total time: ~15-20 hours for all 51 models.

STEP 9: Download your trained adapters BEFORE stopping the pod

Option A: Download via SCP

```
scp -r root@<pod-ip>:~/COPPER/*-lora/final_adapter ./adapters/
```

Option B: Push to HuggingFace Hub

```
huggingface-cli login
for agent in AXIS AEON CYPHER; do
  python finetune_agent.py --agent $agent --merge_and_push \
    --hub_repo your-username/copper-$agent-7b
done
```

STEP 10: STOP THE POD immediately after downloading.

Billing continues until the pod is terminated.

Section 4: STEP-BY-STEP -- TRAINING ON VAST.AI (Cheapest)

STEP 1: Create account at <https://vast.ai>, add credit (\$10-\$20).

STEP 2: Search for GPUs

- * Filter: RTX 3090, 24GB VRAM, Verified hosts only
- * Sort by: Price (lowest first)
- * Pick a host with 99%+ reliability and 100Mbps+ bandwidth

STEP 3: Select a template

- * Use "PyTorch" or "CUDA" pre-installed template
- * Allocate 50GB+ disk space

STEP 4: SSH into the instance

```
ssh -p <port> root@<ip> -i ~/.ssh/id_rsa
```

STEP 5: Same as RunPod Steps 4-9 above

```
Clone/upload code -> pip install -> generate data -> train
```

STEP 6: STOP the instance from the Vast.ai dashboard

(Do NOT just close the terminal -- billing continues!)

IMPORTANT: Vast.ai preserves your disk between stop/start cycles

(billed separately at ~\$0.01/GB/month). Destroy the instance completely once you've downloaded all adapters.

Section 5: STEP-BY-STEP -- TRAINING ON GOOGLE COLAB (Free)

Google Colab provides a free T4 GPU (15GB VRAM). This is enough to train 3B parameter models comfortably, or 7B models with aggressive memory optimization.

STEP 1: Open <https://colab.research.google.com>

Create a new notebook.

STEP 2: Enable GPU runtime

Runtime -> Change runtime type -> GPU -> T4

STEP 3: Install dependencies (run in a cell)

```
!pip install -q trl peft transformers bitsandbytes accelerate
```

STEP 4: Upload your project to Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

STEP 5: Generate datasets

```
%cd /content/drive/MyDrive/COPPER
!python generate_all_agents.py --size 1500 --outdir ./dataset
```

STEP 6: Train ONE agent at a time (free tier has ~12hr sessions)

```
!python finetune_agent.py \
  --agent AXIS \
  --model mistralai/Mistral-7B-Instruct-v0.3 \
  --epochs 3 \
  --batch_size 1 \
  --grad_accum 16 \
  --max_seq_len 512 \
  --bf16
```

STEP 7: Save adapter to Google Drive after each agent

```
!cp -r ./axis-lora/final_adapter /content/drive/MyDrive/COPPER/adapters/AXIS/
```

LIMITATIONS:

- * Free tier disconnects after ~12 hours (or sooner if idle)
- * You can train ~3-4 agents per session
- * Pro (\$10/mo) gives longer sessions and occasional A100 access

Section 6: LOCAL TRAINING -- RTX 5050 8GB (ROG Strix G16 2025)

Your specs:

- * CPU: AMD Ryzen 9 8965HX
- * GPU: NVIDIA RTX 5050 (8GB VRAM)
- * RAM: 16GB or 32GB (GET 32GB -- see below)

WHY 32GB RAM IS ESSENTIAL:

Loading a 7B model in Python spikes system RAM to 16-20GB before the model is quantized and moved to VRAM. With only 16GB total, Windows will page to the SSD, making loading painfully slow and potentially causing OOM crashes during tokenization.

WHAT CAN YOU TRAIN ON 8GB VRAM?

+-----+		
Model Size	Status on RTX 5050 (8GB)	
+-----+		
1.5B (Qwen2.5-1.5B)	EASY -- Trains fast, zero OOM risk	
3B (Phi-3.5-mini)	COMFORTABLE -- QLoRA batch=2, seq=1024	
7B (Mistral-7B)	TIGHT -- Must use all tricks below	
13B+	WILL NOT FIT	
+-----+		

ESSENTIAL TRAINING SETTINGS FOR 8GB VRAM:

```
python finetune_agent.py \
  --agent AXIS \
```

```
--model mistralai/Mistral-7B-Instruct-v0.3 \  
--epochs 3 \  
--batch_size 1 \  
--grad_accum 16 \  
--max_seq_len 512 \  
--lora_r 32 \  
--lora_alpha 64 \  
--fp16
```

Key flags:

batch_size=1	-> Minimum memory usage per step
grad_accum=16	-> Effective batch of 16 via accumulation
max_seq_len=512	-> Halves activation memory vs 1024
lora_r=32	-> Smaller rank = less VRAM
fp16	-> Use fp16 (RTX 5050 supports bf16 too)

RECOMMENDED: USE UNSLOTH LIBRARY

Unsloth reduces training memory by 60-80% and speeds up 2-5x.

```
pip install torch --index-url https://download.pytorch.org/whl/cu121  
pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"  
pip install trl peft transformers bitsandbytes accelerate
```

Section 7: CONVERTING LORA ADAPTERS TO GGUF (Q4 Quantized)

After training, each agent has a LoRA adapter at:

```
./<agent-lowercase>-lora/final_adapter/
```

To run these locally via Ollama, you need to convert them to GGUF
format with 4-bit quantization (Q4_K_M).

METHOD A: Using Unsloth (Easiest -- One Python Call)

```
-----  
  
from unsloth import FastLanguageModel  
  
model, tokenizer = FastLanguageModel.from_pretrained(  
    model_name="./axis-lora/final_adapter",  
    max_seq_length=1024,  
    load_in_4bit=True,  
)
```

Export directly to GGUF Q4

```
model.save_pretrained_gguf(  
    "axis-q4",  
    tokenizer,  
    quantization_method="q4_k_m"  
)
```

Output: axis-q4/unsloth.Q4_K_M.gguf

METHOD B: Manual Pipeline (llama.cpp)

```
Step 1: Merge the LoRA adapter into the base model
python finetune_agent.py --agent AXIS --merge_only \
  --output_dir ./axis-merged

Step 2: Clone llama.cpp and build the converter
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
pip install -r requirements/requirements-convert_hf_to_gguf.txt

Step 3: Convert to GGUF (FP16)
python convert_hf_to_gguf.py ../axis-merged --outfile axis-f16.gguf

Step 4: Quantize to Q4_K_M
./llama-quantize axis-f16.gguf axis-q4.gguf Q4_K_M

Output: axis-q4.gguf (~4.0 GB for a 7B model)
```

METHOD C: Batch Script (All 50 Agents + COPPER)

#!/bin/bash

```
AGENTS="AEGIS AEON AETHER APEX ARGUS ATLAS AXIS BEACON CANVAS
CHRONOS CRUCIBLE CYPHER DIRECTOR ECHO ENIGMA FORGE GLITCH GOLIATH
HAWK HERMES IRIS KINETIC LEDGER LUMEN MNEMONIC NEXUS OMNI ORACLE
PHANTOM PIVOT POLYGLOT PORTAL PRISM PROXY PULSE QUANTA RAPTOR
RENDER SIREN SONAR SPECTRE SPIDER SYNAPSE TALON TENSOR VANGUARD
VAULT VORTEX WARDEN ZENITH COPPER"

mkdir -p gguf_models

for AGENT in $AGENTS; do
  LOWER=$(echo "$AGENT" | tr '[:upper:]' '[:lower:]')
  echo "Converting $AGENT..."

  python -c "
from unsloth import FastLanguageModel
model, tokenizer = FastLanguageModel.from_pretrained(
    './${LOWER}-lora/final_adapter', max_seq_length=1024, load_in_4bit=True)
model.save_pretrained_gguf('gguf_models/${LOWER}', tokenizer,
    quantization_method='q4_k_m')
"

  echo "Done: $AGENT -> gguf_models/${LOWER}/"
done
```

Section 8: LOADING MODELS INTO OLLAMA

Once you have the .gguf files, load them into Ollama so the COPPER backend can call them via API.

STEP 1: Install Ollama

Download from <https://ollama.com/download>
(Windows, macOS, and Linux supported)

STEP 2: Create a Modelfile for each agent

For each agent, create a file like this:

```
--- FILE: axis.Modelfile ---
FROM ./gguf_models/axis/unsloth.Q4_K_M.gguf

TEMPLATE """{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}<|im_start|>user
{{ .Prompt }}<|im_end|>
<|im_start|>assistant
"""

PARAMETER temperature 0.3
PARAMETER top_p 0.9
PARAMETER stop "<|im_end|>"
PARAMETER num_ctx 1024
--- END FILE ---
```

STEP 3: Register with Ollama

```
ollama create copper-axis -f axis.Modelfile
ollama create copper-aeon -f aeon.Modelfile
```

... repeat for all 50 agents + COPPER

STEP 4: Verify it works

```
ollama run copper-axis "Scan my project directory for large files"
```

STEP 5: Batch registration script

#!/bin/bash

```
for AGENT in aegis aeon aether apex argus atlas axis beacon canvas \
chronos crucible cypher director echo enigma forge glitch goliath \
hawk hermes iris kinetic ledger lumen mnemonic nexus omni oracle \
phantom pivot polyglot portal prism proxy pulse quanta raptor \
render siren sonar spectre spider synapse talon tensor vanguard \
vault vortex warden zenith copper; do

    echo "Registering copper-$AGENT..."
    cat > /tmp/${AGENT}.Modelfile << EOF
FROM ./gguf_models/${AGENT}/unsloth.Q4_K_M.gguf
PARAMETER temperature 0.3
PARAMETER num_ctx 1024
PARAMETER stop "<|im_end|>"
EOF

    ollama create copper-${AGENT} -f /tmp/${AGENT}.Modelfile
done
echo "All agents registered with Ollama!"
```

MEMORY USAGE PER MODEL (Q4_K_M):

Base Model	GGUF Size	VRAM Usage (inference)
1.5B Q4	~1.0 GB	~1.2 GB
3B Q4	~2.0 GB	~2.5 GB
7B Q4	~4.0 GB	~4.5-4.8 GB

With your RTX 5050 (8GB VRAM), a 7B Q4 model uses ~4.8GB, leaving ~3.2GB free for the OS and other apps. Only ONE model is loaded at a time (hot-swapped by the COPPER backend).

Section 9: INTEGRATING MODELS INTO THE COPPER PROJECT

The COPPER backend already supports Ollama integration. Here is how the connection works:

- The backend's orchestrator.py routes each user message to an agent (e.g., AXIS) based on keyword matching.
- The backend calls Ollama's local API:

```
POST http://localhost:11434/api/generate
{
  "model": "copper-axis",
  "prompt": "<user message>",
  "stream": false,
  "options": { "num_ctx": 1024 }
}
```
- Ollama loads the agent's Q4 GGUF model into VRAM, generates a response, and unloads it (keep_alive: 0).
- The response is sent back to the frontend via WebSocket.

BACKEND CONFIGURATION:

In backend/app/core/config.py, ensure:

```
OLLAMA_BASE_URL: str = "http://localhost:11434"
OLLAMA_KEEP_ALIVE: str = "0" # Unload after each request
```

In backend/app/data/agents.py, each agent maps to an Ollama model name like "copper-axis", "copper-aeon", etc.

MODEL NAMING CONVENTION:

```
Agent ID -> Ollama Model Name
AXIS     -> copper-axis
AEON     -> copper-aeon
COPPER   -> copper-orchestrator
(pattern: "copper-" + lowercase agent name)
```

Section 10: TROUBLESHOOTING

Problem	Solution
CUDA out of memory during training	Reduce batch_size to 1, reduce max_seq_len to 512, increase grad_accum to 16. Use Unsloth.
Training loss is NaN	Use --bf16 instead of --fp16. If on older GPU (V100), use fp16 with loss scaling enabled.
bitsandbytes import error	Reinstall: pip install bitsandbytes --no-cache-dir Ensure CUDA version matches.
Model download hangs	Pre-download the base model: huggingface-cli download mistralai/Mistral-7B-Instruct
401 Unauthorized on HF	huggingface-cli login Accept the model license on huggingface.co first.
Pod terminated, lost adapters	ALWAYS download or push to Hub before stopping cloud instances
Ollama: "model not found"	Run: ollama list Ensure you ran ollama create.
GGUF conversion fails	Ensure you have enough RAM (16GB+ for 7B models). Close all other applications.
Ollama response is garbage	Check that the Modelfile TEMPLATE matches the model's chat format (ChatML, Alpaca).

Section 11: COST SUMMARY & RECOMMENDATIONS

RECOMMENDED PATH (Budget-Conscious):

- | | |
|--|-------------|
| 1. Generate datasets locally on your current machine | -> \$0 |
| 2. Rent a Vast.ai RTX 3090 for ~17 hours | -> ~\$4-6 |
| 3. Train all 51 models (50 agents + COPPER) | -> Included |
| 4. Download all LoRA adapters to your local machine | -> \$0 |
| 5. Convert LoRA -> GGUF Q4 locally (CPU + 32GB RAM) | -> \$0 |
| 6. Load into Ollama on your RTX 5050 laptop | -> \$0 |

TOTAL: \$4 - \$6 for the entire 51-model fleet.

ALTERNATIVE (Zero Cost):

1. Use Google Colab Free T4 GPU
2. Train 3-4 agents per session (each session ~12 hours)
3. Save adapters to Google Drive between sessions
4. ~13-17 sessions to train all 51 models
5. Convert and load locally as above

TOTAL: \$0 (but requires ~2 weeks of daily Colab sessions)

ALTERNATIVE (Local Only -- After Getting Your Laptop):

1. Use Unsloth on your RTX 5050
2. Train 3B models (comfortable) or 7B models (tight)
3. Each agent takes ~30-60 minutes locally
4. Total: ~25-50 hours of local GPU time

TOTAL: \$0 + electricity

END OF GUIDE

Quick Reference Card

Generate datasets (CPU)

```
python generate_all_agents.py --size 1500 --outdir ./dataset
```

Generate COPPER dataset

```
python copper_orchestrator_dataset_gen.py --size 2500 --outdir ./dataset/COPPER
```

Train all (cloud 24GB)

```
bash launch_finetune_all.sh 24gb
```

Train one agent (8GB)

```
python finetune_agent.py --agent AXIS --batch_size 1 --grad_accum 16 --max_seq_len 512
```

Convert to GGUF Q4

```
model.save_pretrained_gguf('agent-q4', tokenizer, quantization_method='q4_k_m')
```

Register in Ollama

```
ollama create copper-axis -f axis.Modelfile
```

Test model

```
ollama run copper-axis 'Hello'
```

Monitor GPU

```
watch -n 2 nvidia-smi
```